

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. Explica en qué consiste una prueba de cobertura de condiciones y decisiones

Pregunta 2. ¿Cómo se denomina la técnica utilizada en Java para determinar la cobertura de ejecución de un programa (o el grado en el que el código fuente de un programa ha sido probado o ejecutado)?

Pregunta 3. Nombra una herramienta que proporcione andamiaje para pruebas de unidad:

Pregunta 4. ¿Qué es lo que se consigue generar con una técnica de pruebas (o técnica de diseño de pruebas)?

Pregunta 5. Distingue las posibles formas de ejecutar las pruebas

Pregunta 6. Dentro del contexto de la plataforma *Cucumber* y el lenguaje mínimamente controlado *Gherkin*, relaciona las siguientes palabras reservadas con su utilización para describir en texto plano las distintas partes del comportamiento esperado del *software*:

- | | | | |
|----------|----|----|--------------------------------------|
| When | 1● | ●a | Funcionalidad del objeto de pruebas. |
| Feature | 2● | ●b | El estado esperado del sistema |
| Given | 3● | ●c | El cambio real sobre el sistema |
| Then | 4● | ●d | Caso de pruebas concreto. |
| Scenario | 5● | ●e | Precondiciones del sistema a probar |

Pregunta 7. Describe brevemente la tarea de “determinar la estrategia de pruebas” dentro de la elaboración del plan de pruebas maestro

Pregunta 8. Dada la siguiente tabla donde se indica el tamaño de los componentes de un sistema y su clase de riesgo, ¿Cuál sería el número de horas dedicadas a las pruebas de cada componente utilizando una estimación proporcional basada en riesgos si disponemos de 90 horas para dedicar en total al proceso de pruebas y el factor de dedicación para las clases de riesgo A, B y C es 1'5, 1 y 0'5 respectivamente?

Componente	Tamaño	Clase de riesgo	Horas
1	10	A	
2	20	B	
3	20	C	

Pregunta 9. ¿Por qué es útil contar con una organización permanente de pruebas dentro de una empresa de desarrollo de software?

Pregunta 10. ¿Qué se puede automatizar cuando se realizan pruebas de usabilidad? Nombra alguna herramienta que conozcas.

EJERCICIO (2 puntos)

Diseñad pruebas para el siguiente código utilizando la técnica de pruebas de caminos con **nivel de profundidad igual a 3**:

1. Obtened primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código (0,2 puntos).
2. Aplicad la técnica de pruebas de caminos con profundidad 3 para obtener el conjunto de caminos completos correspondiente (0,9 puntos).
3. Finalmente, estableced casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,9 puntos).

```
/**
 * Devuelve un vector de enteros que tiene el mismo número de componentes
 * que el parámetro «vector» y los datos positivos de este ocupando las
 * primeras posiciones. Las últimas componentes del valor devuelto son
 * iguales a 0.
 *
 * @param vector
 *        - el vector del que van a ser copiados sus datos
 *          estrictamente positivos.
 *
 * @return Un vector de enteros que tiene el mismo número de componentes
 *         que el parámetro «vector» y los datos positivos de este
 *         ocupando las primeras posiciones. Las últimas componentes del
 *         valor devuelto son iguales a 0.
 */
public static int[] filtrar(int[] vector) {
    int[] resultado = new int[vector.length];
    A int iR = 0;
    int iV = 0;

    B while (iV < vector.length) {
        C if (vector[iV] > 0) {
            D resultado[iR] = vector[iV];
            iR++;
        }
        E iV++;
    }

    F return resultado;
}
```